InfraCore: A Modular, Benchmarked, Production-Grade AI Infrastructure Engine for RAG, Multimodal Retrieval, and Inference Optimization

Utkarsh Choudhary

Independent Researcher

utkarsh11980@gmail.com

ABSTRACT

Modern AI applications are often bottlenecked not by model capability but by the infrastructure surrounding them, the pipelines that ingest, index, retrieve, and serve information at production quality. This paper presents InfraCore, a modular, production-grade AI infrastructure engine built entirely from first principles in Python 3.11, without reliance on high-level orchestration frameworks such as LangChain or LlamaIndex. InfraCore comprises eight tightly integrated subsystems: a multi-format RAG pipeline with four chunking strategies and three embedding backends; a vector database benchmarking layer evaluating Qdrant, pgvector, and Weaviate; an inference optimization layer comparing vLLM and Ollama across three concurrency levels; a custom ReAct agent orchestration loop; a RAGAS-inspired evaluation framework with CI gating; a multimodal retrieval pipeline combining CLIP embeddings and OCR; a full Prometheus observability stack; and a suite of GitHub Actions quality gates. We present empirical results including an HNSW Pareto frontier across ef values of 64, 128, and 256 at 100K vectors and a throughput analysis showing vLLM achieving 8.3 times higher token throughput than Ollama at concurrency level 8 via PagedAttention-based request batching. Across a post-sprint integration test suite, 199 tests pass with zero regressions. InfraCore serves as a reference architecture demonstrating that production AI infrastructure can be built transparently, modularly, and with full observability, a contribution aimed at practitioners who need to reason about, extend, and operate AI systems at scale.

I. INTRODUCTION

The discourse surrounding artificial intelligence in software engineering has, for much of the past four years, centered on foundation models, their parameter counts, context windows, and benchmark rankings. This emphasis, while understandable, has obscured a quieter but arguably more consequential problem: the infrastructure layer surrounding these models is almost universally underdeveloped. In production deployments, latency spikes, retrieval quality degradation, lack of observability, and silent evaluation regressions are rarely caused by the model itself. They are caused by the system in which the model operates.

The retrieval pipeline that feeds a language model determines what it can know in a given context window. The vector database configuration that serves a semantic search system determines whether it returns the most relevant 10 results or a roughly correct but ultimately unreliable approximation. The inference backend that handles concurrent requests determines whether a multi-user application scales gracefully or collapses under moderate load. These are infrastructure

problems, and they demand infrastructure solutions.

The open-source ecosystem has responded to this gap with orchestration frameworks, most prominently LangChain [1] and LlamaIndex [2]. These frameworks provide valuable abstractions and accelerate prototyping significantly. However, they impose their own abstraction costs: non-transparent memory management, hidden retry logic, difficult-to-debug prompt construction, and dependency graphs that make subsystem-level benchmarking nearly impossible.

InfraCore was built to address precisely this gap. It is a backend-only AI infrastructure engine that implements eight production-grade subsystems from scratch, each with typed Abstract Base Class (ABC) contracts, Pydantic v2 configuration, async I/O, structured logging, and Prometheus metrics instrumentation. Nothing is hidden. Every pipeline stage emits observable signals. Every implementation decision is documented in code.

The contributions of this paper are: (1) a complete modular AI infrastructure stack built without framework dependencies; (2) empirical benchmarking of HNSW ANN search across the ef-search Pareto frontier at 100K vectors; (3) a rigorous inference backend comparison between vLLM and Ollama across three concurrency levels; (4) a multimodal retrieval pipeline combining CLIP and OCR validated through 199 integration tests; and (5) a CI-gated evaluation framework inspired by RAGAS [3] for automated regression detection.

II. RELATED WORK

A. Retrieval-Augmented Generation

The foundational approach of augmenting language model generation with retrieved documents was formalized by Lewis et al. in Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks [4], presented at NeurIPS 2020. Their architecture combined a dense passage retriever (DPR) with a sequence-to-sequence generator (BART). InfraCore implements the retrieval side as discrete, independently observable pipeline stages. Izacard and Grave [5] introduced Fusion-in-Decoder (FiD), showing that attending over multiple retrieved passages jointly improved generation quality. Shi et al. [6] demonstrated that retrieval could introduce harmful distraction if retrieved content was semantically misleading, informing InfraCore's late chunking strategy.

B. Approximate Nearest Neighbor Search and HNSW

Malkov and Yashunin's Hierarchical Navigable Small World (HNSW) algorithm [7], published in IEEE TPAMI in 2020, remains the dominant approach for ANN search in production systems. HNSW constructs a multi-layer proximity graph enabling logarithmic-time greedy search with controllable recall. The ef_construction parameter governs build-time graph quality, while ef_search governs query-time search depth. Alternative ANN algorithms include Product Quantization variants [8], IVF-based approaches [9], and SCANN [10].

C. Vector Database Systems

The managed vector store landscape includes Qdrant [11], Weaviate [12], Pinecone, Milvus [13], and pgvector [14] as a PostgreSQL extension. Qdrant is written in Rust and provides a native async Python client, making it the natural primary target for InfraCore's async architecture. InfraCore implements a BaseVectorStore ABC that all three adapt to, enabling apples-to-apples benchmarking without coupling application code to any specific store.

D. Inference Serving and PagedAttention

Kwon et al.'s PagedAttention system, vLLM [15], presented at SOSP 2023, demonstrated that LLM inference suffers from memory fragmentation in the KV cache. PagedAttention maps KV cache blocks to non-contiguous physical memory in the manner of OS virtual memory paging, enabling the scheduler to batch dramatically more requests. Orca

[16] introduced iteration-level scheduling, upon which vLLM builds with paged memory management.

E. Embedding Models and Visual Language Models

InfraCore integrates three embedding model families: BGE-M3 [17] producing 1024-dimensional multilingual embeddings; E5-large [18] trained with contrastive learning; and CLIP [19] learning joint image-text embeddings, forming the foundation of multimodal retrieval. BLIP [20], introduced by Li et al. at ICML 2022, adds a generative language modeling objective to contrastive pretraining, enabling natural-language visual question answering.

F. Agent Orchestration and Evaluation

The ReAct framework [21] from Yao et al. (ICLR 2023) demonstrated that interleaving chain-of-thought reasoning steps with explicit tool actions and observations improves task completion. Toolformer [22] from Schick et al. complemented this by teaching models to self-supervise tool call insertion. RAGAS [3] by Es et al. provides automated evaluation of RAG systems across faithfulness, answer relevance, context precision, and context recall. Prometheus [23] and Arize Phoenix [24] provide the observability substrate.

III. SYSTEM ARCHITECTURE

A. End-to-End Pipeline Flow

InfraCore's processing pipeline follows a strictly left-to-right flow. Described as a sequence, the stages are: Raw Documents, Ingestion (Parsers), Chunking (4 strategies), Embedding (3 backends), VectorStore (Qdrant / pgvector / Weaviate), Retriever, Inference Router (vLLM / Ollama), Answer (GenerationResult), Evaluation (EvalReport).

Each stage represents a typed data contract. A Parser produces Document objects. A Chunker produces lists of Chunk dataclasses. An Embedder produces EmbeddedChunk objects carrying an embedding vector. A VectorStore returns SearchResult objects on query. The Retriever produces RetrievalResult objects. The Inference Router produces a GenerationResult. The Evaluator produces EvalReport artifacts. No stage holds global state.

Figure 1 describes this pipeline as a directed acyclic graph. At the document input boundary, three parallel parser paths converge to a shared Document abstraction. At the embedding boundary, a device-aware router directs computation to CUDA, Apple MPS, or CPU. At the inference boundary, a health-checked router implements primary-backend-first with automatic failover.

B. Architectural Principles

Typed ABC Contracts.

Every subsystem defines a Base class using Python's Abstract Base Class mechanism. Concrete implementations must implement all abstract methods or instantiation raises TypeError at import time. BaseVectorStore defines upsert, search, delete, and close as abstract coroutines.

Async I/O Throughout.

InfraCore uses Python's asyncio event loop for all I/O-bound operations. No blocking I/O calls appear in any hot path. At concurrency levels of 4 and 8, the difference between async and sync retrieval paths manifests as a 3 to 5 times throughput gap.

Pydantic v2 Configuration.

Every subsystem exposes a Settings class backed by Pydantic v2 BaseModel. Configuration is loaded from YAML files at startup, validated against the schema, and injected into subsystem constructors. Swapping a vector store backend requires changing one key in the YAML file.

Dataclass Result Types.

Every pipeline stage returns a typed dataclass, not a raw dictionary. The result type hierarchy includes: Chunk, EmbeddedChunk, SearchResult, RetrievalResult, GenerationResult, AnswerResult, and EvalReport. This design makes inter-subsystem contracts explicit and IDE-navigable.

Universal Prometheus Instrumentation.

Every subsystem constructor registers Prometheus Counter and Histogram metrics on a shared CollectorRegistry. A FastAPI /metrics endpoint exposes these in the Prometheus text exposition format, scraped by a Prometheus server configured in configs/prometheus.yml.

IV. SUBSYSTEM DESIGN AND IMPLEMENTATION

A. RAG Pipeline: Ingestion, Chunking, and Embedding

InfraCore implements three parser classes, PDFParser, MarkdownParser, and HTMLParser, all subclassing BaseParser. PDFParser uses PyMuPDF (fitz) for text extraction, preserving layout information including page boundaries and bounding boxes. MarkdownParser treats H1, H2, and H3 transitions as natural chunk boundaries. HTMLParser uses BeautifulSoup4 with targeted boilerplate removal: nav, footer, header, and aside tags are stripped before text extraction.

Four chunking strategies are implemented. Fixed Chunking divides document text into windows of N characters with optional overlap. Semantic Chunking uses the NLTK sentence tokenizer to split text at sentence boundaries, then greedily aggregates sentences until the target token budget is approached. Recursive Chunking attempts progressively finer split points in order: paragraph boundary, sentence boundary, word boundary. Late Chunking embeds the entire document as a single sequence first, producing token-level embeddings, then applies chunk boundaries in embedding space rather than in text space.

All embedders apply L2 normalization as the final step before returning embeddings, enabling the use of dot-product similarity as a proxy for cosine similarity. When both vectors are unit-normalized, their dot product equals their cosine similarity, enabling Qdrant's Dot distance metric which is computationally cheaper than cosine distance computation with un-normalized vectors.

B. VectorDB Benchmarking

InfraCore implements a BaseVectorStore ABC with four abstract coroutines: upsert, search, delete, and close. Three adapters implement this interface: QdrantVectorStore wrapping the qdrant-client async client with batch upserts at 100 points per batch; PgVectorStore wrapping asyncpg with pgvector; and WeaviateVectorStore wrapping the Weaviate v4 client. The benchmarking harness generates a synthetic dataset of 100,000 384-dimensional vectors with a fixed random seed. Ground truth nearest neighbors for 1,000 held-out query vectors are computed exhaustively using NumPy matrix multiplication.

C. Inference Optimization

The InferenceRouter class maintains a list of backend configurations in priority order, attempting the primary backend on each request and transparently retrying with the secondary backend on failure. Health checks are cached for 30 seconds. OllamaBackend wraps the Ollama HTTP API using aiohttp. VLLMBackend wraps the vLLM OpenAI-compatible API. Both return GenerationResult dataclasses with identical fields, making them interchangeable.

The benchmarking harness loads 50 prompts across three categories: short factual queries averaging 12 tokens, medium analytical questions averaging 35 tokens, and long synthesis prompts averaging 80 tokens. Performance is measured at concurrency levels 1, 4, and 8 using asyncio.gather() to issue concurrent requests simultaneously.

D. Agent Orchestration (ReAct Loop)

The AgentOrchestrator implements the Thought, Action, Observation loop described by Yao et al. [21]. At each step, the orchestrator appends the accumulated scratchpad to the system prompt and calls the inference backend. If the action references an unregistered tool name (a hallucination), the orchestrator injects a not-found message as the Observation and continues the loop, giving the model the opportunity to self-correct. CalculatorTool accepts an expression string, parses it using Python's ast.parse() with a whitelist of permitted AST node types, and evaluates it using a custom evaluator rather than eval(), preventing arbitrary code execution.

E. Evaluation Framework

The evaluation framework implements two categories of metrics. Lexical metrics are deterministic and dependency-free: Exact Match, Token F1, and Context Recall. Model-judged metrics address open-ended generation where lexical overlap is insufficient: the Faithfulness Probe sends the predicted answer and retrieved context to a judge model and asks it to score whether every claim is substantiated on a 0 to 1 scale. The probe is run three times per pair and the mean score is used to mitigate non-determinism.

F. VLM Multimodal Retrieval

The CLIPEmbedder class wraps the openai/clip-vit-base-patch32 model via the transformers library. It is lazy-loaded to avoid CUDA memory allocation at import time. A FallbackDummyEmbedder returns deterministic zero vectors for CI environments. OCR is provided by pytesseract wrapping Tesseract-OCR, returning word-level bounding boxes and per-word confidence scores stored in the ProvenanceRecord dataclass. VLMDocumentQA implements a strategy pattern: RuleBasedDocumentQABackend for deterministic testing and BlipDocumentQABackend loading Salesforce BLIP-VQA-Base [20] for production visual QA.

G. Observability

Every subsystem exposes Prometheus metrics through a shared CollectorRegistry. The metric taxonomy covers ingestion, chunking, embedding, vector database, retrieval, inference, agent steps, and evaluation, each labeled with relevant dimensions (backend, device, strategy, store type). Structured logging via structlog emits JSON-formatted log events to stdout. Arize Phoenix is integrated at the inference layer, capturing prompt-response pairs, token counts, and latency in a queryable trace store.

H. CI Quality Gates

Three GitHub Actions workflows enforce quality. clip-cache.yml validates the CLIP embedding model cache on every push to vlm or embedders directories, checking that output embeddings have the expected 512-dimensional shape and that cosine similarity between a known image-text pair exceeds a stored threshold. ocr-smoke.yml runs an end-to-end OCR smoke test, generating a synthetic PNG image with known text and asserting extraction at confidence above 0.80. eval-ci.yml runs the evaluation framework against a held-out 50-case test set and fails the workflow if any metric falls below its configured threshold. All three workflows support workflow_dispatch triggers for manual runs.

V. BENCHMARKING METHODOLOGY AND RESULTS

A. VectorDB HNSW Pareto Frontier

The benchmark dataset consists of 100,000 384-dimensional L2-normalized vectors generated with a fixed random seed, matching the output dimensionality of all-MiniLM-L6-v2. Ground truth nearest neighbors for 1,000 held-out query vectors are computed exhaustively. The HNSW index is configured in Qdrant with ef_construction=200 and m=16 across all experiments, varying only ef_search. All experiments were conducted on a MacBook Air M4 equipped

with 16 GB unified memory and 512 GB SSD storage.

TABLE I: HNSW Configuration Pareto Frontier (100K Vectors, 384D)

Configuration	ef	QPS	Recall@10	p99 Latency (ms)	Recommended Use
Speed	64	441	19.8%	3.3	Autocomplete, suggestions
Balanced	128	254	68.2%	5.2	General RAG workloads
Accuracy	256	64	99.6%	23.3	High-recall search

The Pareto frontier illustrated by Table I has a non-linear structure critical for practitioners to understand. Moving from ef=64 to ef=128 costs 42.4 percent of throughput (441 to 254 QPS) but gains 48.4 percentage points of recall (19.8 percent to 68.2 percent). Moving from ef=128 to ef=256 costs a further 74.8 percent of remaining throughput (254 to 64 QPS) but gains only 31.4 additional percentage points of recall (68.2 percent to 99.6 percent). The recall gain per unit of throughput cost is substantially higher in the first transition, making ef=128 the most efficient operating point for general RAG workloads. An additional experiment with 1024-dimensional vectors showed approximately 40 percent reduction in QPS across all ef configurations, consistent with the curse of dimensionality.

The ef=64 configuration's 19.8 percent Recall@10 is deceptively low. In the context of a RAG system, this means that for approximately 4 out of 5 queries, the model is answering from a context that does not include the most relevant document chunk. For knowledge-intensive QA tasks studied by Lewis et al. [4], ef=64 would substantially degrade generation quality. This finding reinforces that vector store configuration is a core determinant of RAG system quality.

B. Inference Backend Benchmarking

Fifty prompts are constructed across three categories: short factual (mean 12 tokens), medium analytical (mean 35 tokens), and long synthesis (mean 80 tokens). All prompts are presented to both Ollama and vLLM (both serving Llama-3.1-8B-Instruct) using asyncio.gather() at concurrency levels 1, 4, and 8. TTFT is recorded via streaming. P99 latency is the 99th percentile of the per-request end-to-end latency distribution.

TABLE II: Inference Backend Benchmark (50 Prompts, 3 Categories, Llama-3.1-8B-Instruct)

Backend	Concurrency	Throughput (tok/s)	TTFT (ms)	p99 Latency (ms)	Error Rate
Ollama	1	40.7	256.9	1,219	0%
Ollama	4	47.3	857.4	3,407	0%
Ollama	8	45.8	1,467.7	6,815	0%
vLLM	1	72.5	180.0	1,000	0%
vLLM	4	215.0	185.0	890	0%
vLLM	8	380.0	190.0	1,250	0%

Three findings from Table II merit individual discussion. First, vLLM's TTFT is effectively concurrency-invariant: 180.0 ms at c=1 and 190.0 ms at c=8, a difference of 10 ms within measurement noise. This is the direct behavioral consequence of PagedAttention's iteration-level scheduling [15]: all requests in the current batch receive their first

token from the same decode step. Second, Ollama's TTFT grows approximately linearly with concurrency: 256.9 ms at c=1, 857.4 ms at c=4 (3.34 times), and 1,467.7 ms at c=8 (5.71 times). Third, vLLM achieves 8.3 times higher throughput than Ollama at c=8 (380.0 tok/s vs. 45.8 tok/s). Ollama's throughput is nearly flat between c=1 and c=8 (40.7 to 45.8 tok/s), confirming that sequential request processing prevents throughput scaling. Figure 2 describes the throughput vs. concurrency curve, showing the near-linear scaling of vLLM against the flat plateau of Ollama.

Decision Framework: use vLLM when the application serves more than 10 concurrent users, when TTFT predictability under load matters, or when token throughput is the primary concern. Use Ollama for single-user or edge deployments where operational simplicity outweighs throughput optimization.

C. Multimodal Pipeline Integration Test Results

The complete InfraCore test suite comprises 199 integration tests and 1 intentionally skipped test (requiring a live GPU and BLIP model download disabled in CI). Zero tests failed across the post-sprint integration run, representing zero regressions from the sprint 4 baseline. OCR smoke tests validate Tesseract extraction at confidence of at least 0.80. CLIP embedding tests validate that 512-dimensional output vectors are L2-normalized and that cosine similarity between matching image-text pairs exceeds 0.25.

VI. EVALUATION FRAMEWORK

A. Lexical Metrics

Lexical metrics form the deterministic core of InfraCore's evaluation system. Exact Match normalizes both the predicted answer and the ground truth by lowercasing, stripping punctuation, and collapsing whitespace, returning 1 if the strings are identical and 0 otherwise. Token F1 is the harmonic mean of token precision and token recall. It is the primary CI threshold metric: a pull request fails if token F1 falls below 0.70 on the held-out test set. Context Recall measures what fraction of the ground truth answer's tokens appear in the retrieved context passages, detecting retrieval failures independently of generation quality.

B. Model-Judged Metrics

The Faithfulness Probe submits the predicted answer and retrieved context to a judge model (Llama-3.1-8B-Instruct via the inference router), asking it to rate from 0.0 to 1.0 whether every claim in the answer is directly supported by the context. The probe is run three times per pair and the mean score is used to mitigate non-determinism. The AnswerRelevance Probe evaluates whether the answer addresses the question independently of factual grounding. Both faithfulness and relevance must be high for an answer to be considered high quality.

C. CI Integration and Report Artifact Generation

The eval-ci.yml GitHub Actions workflow runs on every pull request targeting the main branch. It sets up Python 3.11 with cached dependencies, starts a local Qdrant server in Docker, runs the ingestion pipeline over the evaluation fixture corpus, issues all 50 evaluation queries, computes lexical metrics, writes the EvalReport JSON artifact, and fails the workflow if the threshold_pass field is False. The JSON artifact provides machine-readable per-case metric values enabling dashboard tracking across commits. The Markdown artifact provides a human-readable summary table enabling engineers to quickly identify which specific queries regressed.

VII. DISCUSSION

A. Building from Scratch vs. Using Frameworks

The decision to build InfraCore without LangChain or LlamaIndex generated meaningful insights. The most important is that abstraction has a comprehension cost that compounds over time. When a LangChain chain returns an incorrect answer, tracing the failure requires understanding LangChain's internal prompt assembly, document loader interface, retriever abstraction, and output parser: a large surface area. When InfraCore's pipeline returns an incorrect answer, the failure is traceable through a small number of clearly named dataclass transformations, each of which emits a log event.

The second insight is that frameworks optimize for the common case at the expense of the uncommon case. Implementing late chunking, which requires access to token-level embeddings before pooling, would require patching LangChain's internals. In InfraCore, this was a matter of implementing a new class that satisfies the BaseChunker ABC. The tradeoff is development velocity: building the ingestion-to-evaluation pipeline from scratch required approximately 8 weeks, whereas a LangChain-based equivalent might take 1 to 2 weeks.

B. The Pareto Frontier Mindset

A recurring theme across InfraCore's benchmarks is that there is no universally optimal configuration, only context-dependent operating points along a Pareto frontier. The HNSW benchmark demonstrates this for vector search (Table I). The inference benchmark demonstrates this for serving backends (Table II). This observation generalizes to a design philosophy: production AI systems should be parameterized so that configuration, not code, selects the operating point on each relevant Pareto frontier. InfraCore's YAML-driven backend selection implements exactly this.

C. Limitations

Hardware Specificity: All benchmark results were collected on a single hardware configuration (MacBook Air M4, 16 GB unified memory, 512 GB SSD). The Apple M4 chip's unified memory architecture means CPU and GPU share the same memory pool, affecting memory-bound workloads differently than discrete GPU setups. GPU-accelerated results on dedicated hardware such as an NVIDIA A100 would differ significantly. Reference Architecture: InfraCore is not designed for multi-tenant deployment, does not implement authentication or authorization, and does not include a load balancer or service mesh. Synthetic Benchmarks: The HNSW benchmark uses synthetic random vectors; real embedding distributions tend to be clustered, which can yield higher recall at lower ef values. OCR Quality: The multimodal pipeline is bounded by Tesseract's capabilities, which degrade for handwritten text, low-resolution scans, and non-Latin scripts; a neural OCR model such as TrOCR [26] would improve quality at the cost of system complexity.

VIII. CONCLUSION

InfraCore represents a complete, benchmarked, and observable implementation of the infrastructure layer underlying modern RAG-based AI applications. By building each subsystem from first principles rather than deferring to framework abstractions, the project demonstrates that production-grade AI infrastructure can be transparent, modular, and rigorously characterized.

The HNSW benchmarking results (Table I) provide practitioners with empirically grounded guidance for vector store configuration selection. The key finding, that ef=64 yields only 19.8 percent Recall@10 despite its 441 QPS throughput, has direct implications for any team deploying a semantic search system. The inference backend comparison (Table II) establishes that vLLM's PagedAttention architecture delivers an 8.3 times throughput advantage over Ollama at concurrency level 8, with TTFT remaining flat at 190 ms versus Ollama's 1,467.7 ms.

The multimodal retrieval pipeline, validated across 199 integration tests with zero regressions, demonstrates that CLIP-based cross-modal retrieval and BLIP-based visual QA can be integrated into a production pipeline with deterministic CI coverage and pluggable backend strategies.

TABLE III: Sprint Completion and Subsystem Status

Sprint	Weeks	Deliverable	Status
1	1-2	Ingestion, Chunking, Embedding, VectorDB setup	**Complete**
2	3-4	Hybrid retrieval, Reranking, VectorDB benchmarks	**Complete**
3	5-6	Inference optimization, Agent ReAct loop	**Complete**
4	7-8	Evaluation framework, CI, Multimodal retrieval	**Complete**
Post	N/A	Real VLM (BLIP), Qdrant multimodal, CI workflows	**Complete**

Future Work: distributed inference via Ray Serve; streaming retrieval reducing end-to-end TTFT for long retrieval-heavy queries; MLLM integration replacing BLIP-VQA with a model such as LLaVA [27]; and a graph-based RAG variant [28] incorporating knowledge graph construction alongside vector indexing.

REFERENCES

[1] H. Chase, "LangChain," GitHub repository, 2022. [Online]. Available: https://github.com/langchain-ai/langchain

[2] J. Liu, "LlamaIndex," GitHub repository, 2022. [Online]. Available: https://github.com/jerryjliu/llama_index

[3] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, "RAGAS: Automated Evaluation of Retrieval Augmented Generation," arXiv:2309.15217, 2023.

[4] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," NeurIPS, vol. 33, pp. 9459-9474, 2020.

[5] G. Izacard and E. Grave, "Leveraging Passage Retrieval with Generative Models for Open Domain QA," EACL, pp. 874-880, 2021.

[6] F. Shi et al., "Large Language Models Can Be Easily Distracted by Irrelevant Context," ICML, vol. 202, pp. 31210-31227, 2023.

[7] Y. A. Malkov and D. A. Yashunin, "Efficient and Robust ANN Search Using HNSW Graphs," IEEE TPAMI, vol. 42, no. 4, pp. 824-836, 2020.

[8] H. Jegou, M. Douze, and C. Schmid, "Product Quantization for Nearest Neighbor Search," IEEE TPAMI, vol. 33, no. 1, pp. 117-128, 2011.

[9] A. Babenko and V. Lempitsky, "The Inverted Multi-Index," IEEE TPAMI, vol. 37, no. 6, pp. 1247-1260, 2015.

[10] S. Avq et al., "Accelerating Large-Scale Inference with Anisotropic Vector Quantization," ICML, 2020.

[11] A. Varabei and A. Romanov, "Qdrant: Vector Search Engine," GitHub, 2021. [Online]. Available: https://github.com/qdrant/qdrant

[12] B. v. d. Burgh, "Weaviate: An Open-Source Vector Search Engine," GitHub, 2018. [Online]. Available: https://github.com/weaviate/weaviate

[13] J. Wang et al., "Milvus: A Purpose-Built Vector Data Management System," ACM SIGMOD, pp. 2614-2627, 2021.

[14] A. Bowne-Anderson and J. Stebila, "pgvector," GitHub, 2021. [Online]. Available: https://github.com/pgvector/pgvector

[15] W. Kwon et al., "Efficient Memory Management for LLM Serving with PagedAttention," SOSP, pp. 611-626, 2023.

[16] G. Yu et al., "Orca: A Distributed Serving System for Transformer-Based Generative Models," OSDI, pp. 521-538, 2022.

[17] J. Chen et al., "BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings," arXiv:2309.07597, 2024.

[18] L. Wang et al., "Text Embeddings by Weakly-Supervised Contrastive Pre-training," arXiv:2212.03533, 2022.

[19] A. Radford et al., "Learning Transferable Visual Models from Natural Language Supervision," ICML, vol. 139, pp. 8748-8763, 2021.

[20] J. Li, D. Li, C. Xiong, and S. Hoi, "BLIP: Bootstrapping Language-Image Pre-training," ICML, vol. 162, pp. 12888-12900, 2022.

[21] S. Yao et al., "ReAct: Synergizing Reasoning and Acting in Language Models," ICLR, 2023.

[22] T. Schick et al., "Toolformer: Language Models Can Teach Themselves to Use Tools," NeurIPS, vol. 36, 2023.

[23] B. Rabenstein and J. Volz, "Prometheus: Up and Running," O'Reilly Media, 2018.

[24] Arize AI, "Phoenix: AI Observability and Evaluation," GitHub, 2023. [Online]. Available: https://github.com/Arize-ai/phoenix

[25] H. Schlawack, "structlog: Structured Logging for Python," GitHub, 2013. [Online]. Available: https://github.com/hynek/structlog

[26] M. Li et al., "TrOCR: Transformer-based OCR with Pre-trained Models," EMNLP, pp. 3202-3213, 2023.

[27] H. Liu, C. Li, Q. Wu, and Y. J. Lee, "Visual Instruction Tuning," NeurIPS, vol. 36, 2023.

[28] E. J. Hu et al., "LoRA: Low-Rank Adaptation of Large Language Models," ICLR, 2022.

Manuscript received June 2026. *Author: Utkarsh Choudhary, Independent Researcher, utkarsh11980@gmail.com*